

# LABS TOC

1. Write a program in C / C++ that accepts your alphabet symbols and a string from that alphabet as input and verify whether the string is from the input alphabet or not. Also print all the suffixes and prefixes of the input string.

**Program:**

```
#include <iostream>
#include <string>
#include <set>
using namespace std;

bool isFromAlphabet(const string& str, const string& alphabet) {
    set<char> alphabetSet(alphabet.begin(), alphabet.end());
    for (char c : str) {
        if (alphabetSet.find(c) == alphabetSet.end()) {
            return false;
        }
    }
    return true;
}

void printSuffixesAndPrefixes(const string& str) {
    cout << "Suffixes:" << endl;
    for (int i = 0; i < str.length(); i++) {
        cout << str.substr(i) << endl;
    }

    cout << "Prefixes:" << endl;
    for (int i = 1; i <= str.length(); i++) {
        cout << str.substr(0, i) << endl;
    }
}

int main() {
    string alphabet, input;

    cout << "Enter alphabet symbols (no spaces): ";
    cin >> alphabet;
```

```

    cout << "Enter a string: ";
    cin >> input;

    if (isFromAlphabet(input, alphabet)) {
        cout << "The string is from the input alphabet." << endl;
        printSuffixesAndPrefixes(input);
    } else {
        cout << "The string is not from the input alphabet." << endl;
    }

    return 0;
}

```

#### Sample Output:

Enter alphabet symbols (no spaces): abc

Enter a string: abac

The string is from the input alphabet.

Suffixes:

abac

bac

ac

c

Prefixes:

a

ab

aba

abac

Enter alphabet symbols (no spaces): abc

Enter a string: abcd

The string is not from the input alphabet.

**2. Write a program in C or C++ that accepts a string and print the substrings as below.  
Construct function for each of these task below in your program.**

- a. The leftmost substring of length n, where n is your input.
- b. The rightmost substring of length n, where n is your input.
- c. The substring from position n to m symbols where m and n are input

**Program:**

```
#include <iostream>
#include <string>
using namespace std;

string leftmostSubstring(const string& str, int n) {
    return str.substr(0, n);
}

string rightmostSubstring(const string& str, int n) {
    return str.substr(str.length() - n);
}

string substringFromNtoM(const string& str, int n, int m) {
    return str.substr(n, m - n + 1);
}

int main() {
    string input;
    int n, m;

    cout << "Enter a string: ";
    cin >> input;

    cout << "Enter n for leftmost and rightmost substrings: ";
    cin >> n;
    cout << "Leftmost substring of length " << n << ": "
        << leftmostSubstring(input, n) << endl;
    cout << "Rightmost substring of length " << n << ": "
```

```

        << rightmostSubstring(input, n) << endl;

    cout << "Enter n and m for substring from n to m: ";
    cin >> n >> m;
    cout << "Substring from position " << n << " to " << m << ": "
        << substringFromNtoM(input, n, m) << endl;

    return 0;
}

```

#### Sample Output:

```

Enter a string: hello
Enter n for leftmost and rightmost substrings: 2
Leftmost substring of length 2: he
Rightmost substring of length 2: lo
Enter n and m for substring from n to m: 1 3
Substring from position 1 to 3: ell

```

### 3. Write a program for List out the All Prefix of a given String.

#### Program:

```

#include <iostream>
#include <string>
using namespace std;

void printAllPrefixes(const string& str) {
    cout << "All Prefixes:" << endl;
    for (int i = 1; i <= str.length(); i++) {
        cout << str.substr(0, i) << endl;
    }
}

int main() {
    string input;

```

```

    cout << "Enter a string: ";
    cin >> input;

    printAllPrefixes(input);

    return 0;
}

```

**Sample Output:**

```

Enter a string: code
All Prefixes:
c
co
cod
code

```

**4. Construct a DFA accepting strings ending at b, from alphabet {a,b}. Construct its transition diagram and implement your DFA in a program in C or C++.**

**Program:**

```

#include <iostream>
#include <string>
using namespace std;

bool acceptDFA(const string& str) {
    bool state = false; // false: not accepted, true: accepted
    for (char c : str) {
        if (c == 'a') state = false;
        else if (c == 'b') state = true;
        else return false; // invalid input
    }
    return state;
}

```

```

}

int main() {
    string input;

    cout << "Enter a string (only 'a' and 'b' allowed): ";
    cin >> input;

    if (acceptDFA(input)) {
        cout << "Accepted: The string ends with 'b'." << endl;
    } else {
        cout << "Rejected: The string does not end with 'b' or
        contains invalid characters." << endl;
    }

    return 0;
}

```

### Sample Output:

```

Enter a string (only 'a' and 'b' allowed): abb
Accepted: The string ends with 'b'.

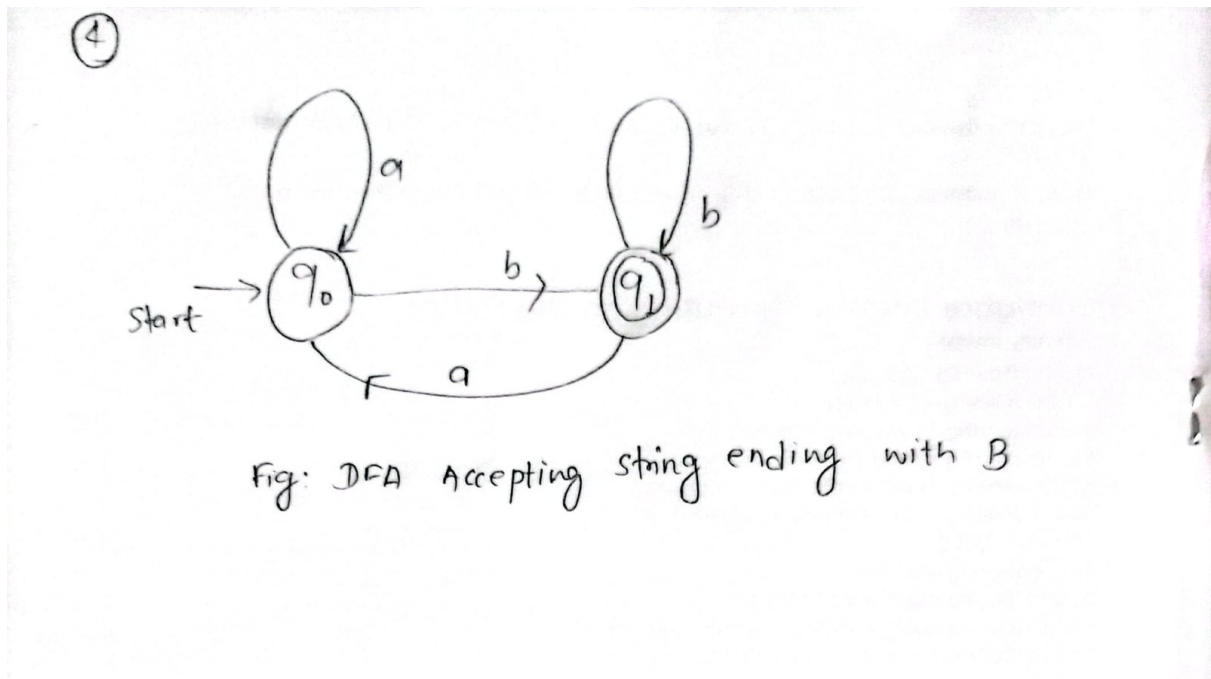
```

```

Enter a string (only 'a' and 'b' allowed): aba
Rejected: The string does not end with 'b' or contains invalid characters.

```

### Designing DFA:



**5. Construct transition table for a DFA accepting all strings from  $\{0,1\}^*$  ending with 01. Write a program in C or C++.**

**Program:**

```

#include <iostream>
#include <string>
using namespace std;

bool acceptDFA(const string& str) {
    int state = 0;
    for (char c : str) {
        switch (state) {
            case 0:
                state = (c == '0') ? 1 : 0;
                break;
            case 1:
                state = (c == '0') ? 1 : 2;
                break;
            case 2:
                state = (c == '0') ? 1 : 0;
                break;
        }
    }
}

```

```

    }
    return state == 2;
}

int main() {
    string input;

    cout << "Enter a string (only '0' and '1' allowed): ";
    cin >> input;

    if (acceptDFA(input)) {
        cout << "Accepted: The string ends with '01'." << endl;
    } else {
        cout << "Rejected: The string does not end with '01'." << endl;
    }

    return 0;
}

```

#### Sample Output:

```

Enter a string (only '0' and '1' allowed): 1001
Accepted: The string ends with '01'.

```

```

Enter a string (only '0' and '1' allowed): 1010
Rejected: The string does not end with '01'.

```

#### Designing DFA:



5

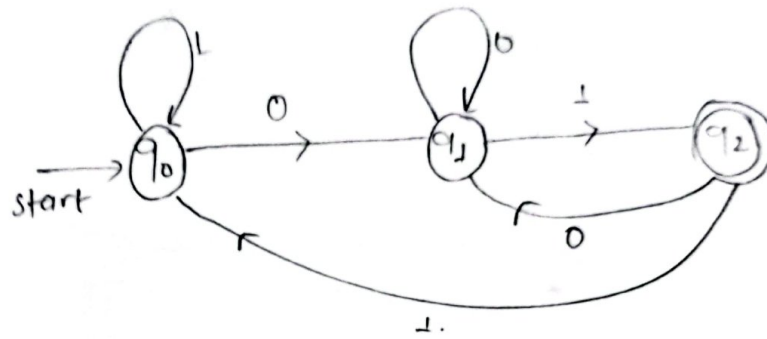


Fig: DFA accepting string ending with 01

Transition table

| State            | 0              | 1              |
|------------------|----------------|----------------|
| → q <sub>0</sub> | q <sub>1</sub> | q <sub>0</sub> |
| q <sub>1</sub>   | q <sub>1</sub> | q <sub>2</sub> |
| * q <sub>2</sub> | q <sub>1</sub> | q <sub>0</sub> |

## 6. Write a Program to validate C identifiers and keywords.

**Program:**

```

#include <iostream>
#include <string>
#include <set>
#include <cctype>
using namespace std;

set<string> keywords = {"int", "float", "char", "double",
                        "if", "else", "while", "
bool isValidIdentifier(const string& str) {
    if (str.empty() || isdigit(str[0])) return false;

```

```

    for (char c : str) {
        if (!isalnum(c) && c != '_') return false;
    }
    return true;
}

int main() {
    string input;

    cout << "Enter a string to validate: ";
    cin >> input;

    if (keywords.find(input) != keywords.end()) {
        cout << "\"" << input << " is a C keyword." << endl;
    } else if (isValidIdentifier(input)) {
        cout << "\"" << input << " is a valid C identifier." << endl;
    } else {
        cout << "\"" << input << " is neither a valid C identifier nor a keyword." << endl;
    }

    return 0;
}

```

#### Sample Output:

```

Enter a string to validate: int
'int' is a C keyword.

```

```

Enter a string to validate: myVar123
'myVar123' is a valid C identifier.

```

```

Enter a string to validate: 123invalid
'123invalid' is neither a valid C identifier nor a keyword.

```

**7. Construct a DFA – Transition graph and Transition table , accepting strings from alphabet {0,1} having substring 010. Write a program in C or C++ to Simulate this DFA.**

**Program:**

```
#include <iostream>
#include <string>
using namespace std;

bool acceptDFA(const string& str) {
    int state = 0;
    for (char c : str) {
        switch (state) {
            case 0:
                state = (c == '0') ? 1 : 0;
                break;
            case 1:
                state = (c == '1') ? 2 : 1;
                break;
            case 2:
                state = (c == '0') ? 3 : 0;
                break;
            case 3:
                break;
        }
        if (state == 3) return true;
    }
    return false;
}

int main() {
    string input;

    cout << "Enter a string (only '0' and '1' allowed): ";
    cin >> input;

    if (acceptDFA(input)) {
        cout << "Accepted: The string contains '010'." << endl;
    } else {
        cout << "Rejected: The string does not contain '010'." << endl;
    }

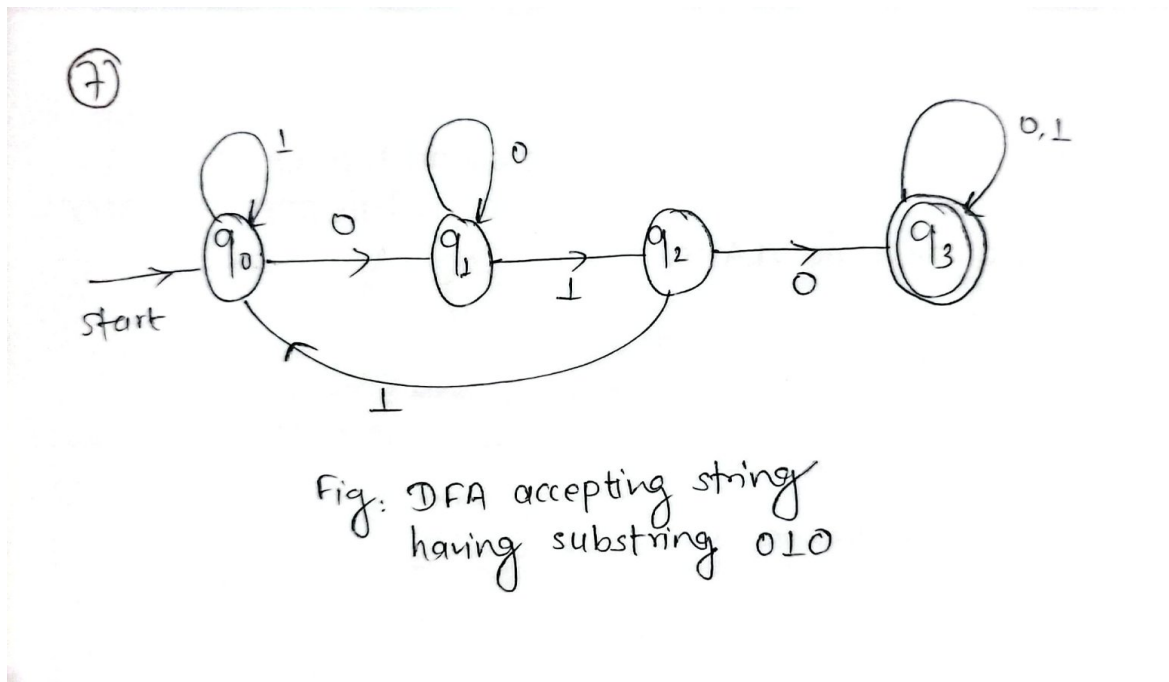
    return 0;
}
```

### Sample Output:

```
Enter a string (only '0' and '1' allowed): 11010
Accepted: The string contains '010'.
```

```
Enter a string (only '0' and '1' allowed): 1100
Rejected: The string does not contain '010'.
```

### Designing DFA:



## 8.Design NFA for accepting strings over alphabet {0,1} starting with 01 and write program in C/C++.

### Program:

```
#include <iostream>
#include <string>
using namespace std;

bool acceptNFA(const string& str) {
    if (str.length() < 2) return false;
    return (str[0] == '0' && str[1] == '1');
}
```

```

}

int main() {
    string input;

    cout << "Enter a string (only '0' and '1' allowed): ";
    cin >> input;

    if (acceptNFA(input)) {
        cout << "Accepted: The string starts with '01'." << endl;
    } else {
        cout << "Rejected: The string does not start with '01'." << endl;
    }

    return 0;
}

```

#### Sample Output:

```

Enter a string (only '0' and '1' allowed): 0110
Accepted: The string starts with '01'.

```

```

Enter a string (only '0' and '1' allowed): 1010
Rejected: The string does not start with '01'.

```

#### Designing NFA:

8

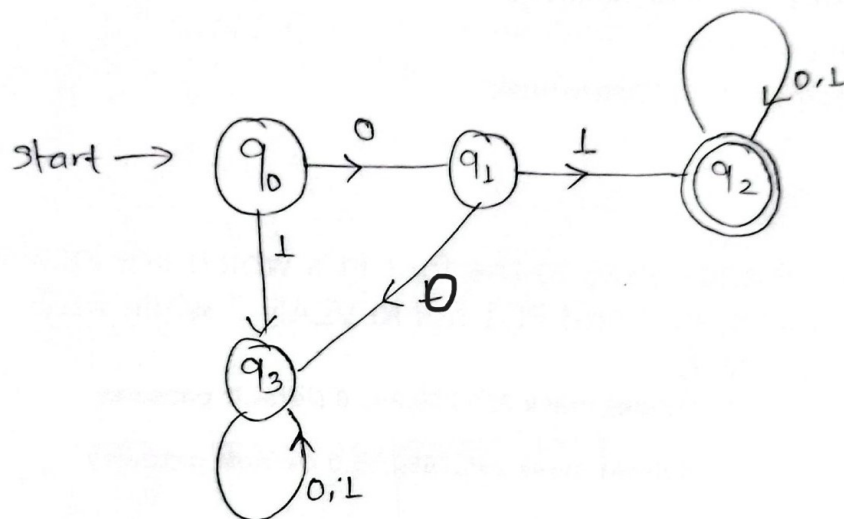


Fig: NFA accepting string starting with 01.

**9. Write a program to simulate a PDA accepting a language of strings over alphabet {0,1} with equal no of 0s and 1s.**

**Program:**

```
#include <iostream>
#include <string>
#include <stack>
using namespace std;

bool acceptPDA(const string& str) {
    stack<char> pda;
    for (char c : str) {
        if (c == '0') {
            pda.push('0');
        } else if (c == '1') {
            if (pda.empty() || pda.top() == '1') {
                pda.push('1');
            } else {
                pda.pop();
            }
        }
    }
}
```

```

    }
}
return pda.empty();
}

int main() {
    string input;

    cout << "Enter a string (only '0' and '1' allowed): ";
    cin >> input;

    if (acceptPDA(input)) {
        cout << "Accepted: The string has an equal number of 0s
            and 1s." << endl;
    } else {
        cout << "Rejected: The string does not have an equal number
            of 0s and 1s." << endl;
    }

    return 0;
}

```

#### Sample Output:

```

Enter a string (only '0' and '1' allowed): 001101
Accepted: The string has an equal number of 0s and 1s.

```

```

Enter a string (only '0' and '1' allowed): 00110
Rejected: The string does not have an equal number of 0s and 1s.

```

### 10. Construct a PDA accepting language $L = \{0^n 1^n \mid n \geq 1\}$ . Write a program to implement this PDA.

#### Code

```

#include <iostream>
#include <stack>
#include <string>

using namespace std;

```

```

bool isValid(string input) {
    stack<char> s;

    for (char ch : input) {
        if (ch == '0') {
            s.push('0');
        } else if (ch == '1') {
            if (s.empty()) return false;
            s.pop();
        }
    }

    return s.empty();
}

int main() {
    string inputs[] = {"0011", "01", "000111", "10", "1100"};

    for (string input : inputs) {
        if (isValid(input)) {
            cout << "Input: " << input << " is Accepted" << endl;
        } else {
            cout << "Input: " << input << " is Rejected" << endl;
        }
    }

    return 0;
}

```

#### Sample Output:

```

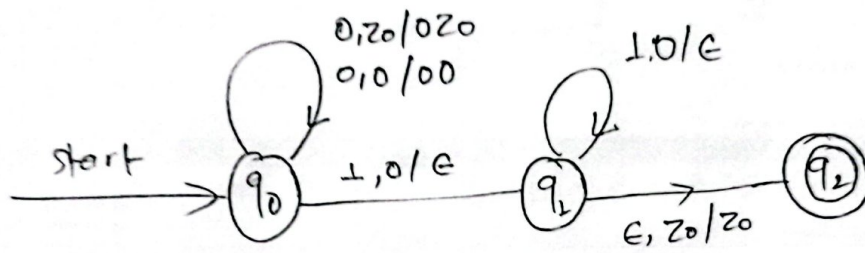
Input: 0011 is Accepted
Input: 01 is Accepted
Input: 000111 is Accepted
Input: 10 is Rejected
Input: 1100 is Rejected

```

#### PDA Construction:



⑤



$$\delta(q_0, 0, z_0) = (q_0, 0z_0)$$

$$\delta(q_0, 0, 0) = (q_0, 00)$$

$$\delta(q_0, 1, 0) = (q_1, \epsilon)$$

$$\delta(q_1, 1, 0) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, z_0) = (q_2, z_0)$$

Fig. PDA accepting  $L = 0^n 1^n$  ( $n \geq 1$ )